

Argus: Multi-Agent / Multi-Model Upgrade

Proposed plan — taking Argus beyond Claude Code, and beyond high-end hardware · 2026-07-26

Context

Argus's AI surface is currently hard-wired to Claude in two independent places:

1. **App features** — chat, the day planner, and study-guide/exam generation (`backend/agent/runtime.py`, `planner.py`, `generate.py`) all call `claude_agent_sdk` directly with a hardcoded model, authenticated via the user's Claude subscription login. A model registry already exists (`backend/config.py`, `.argus/models.json`, the `/system` MODELS panel, `ModelSelect.tsx`) and the frontend already sends the selected model on every chat call — but the backend explicitly rejects any non-Anthropic provider today. It's a fully-built on-ramp with no engine behind it.
2. **Dev-loop journaling** (`claude-integration/`) — deterministic PowerShell hooks that journal coding sessions on the Argus repo itself into the vault (`90-Meta/sessions/...`). Claude-Code-specific only in the shape of the stdin JSON they read and the output they emit.

The goal grew, through discussion, into four related but distinct pieces of work, all under one theme: Argus shouldn't require Claude specifically, and shouldn't require high-end hardware either.

- **Part 1** — pluggable models for chat/planner/study (local, hosted-API, and Codex/Gemini as model choices).
- **Part 1b** — a hardware-aware local model picker (recommend + one-click install), so "run it locally" doesn't require the user to already know what fits their machine.
- **Part 2** — multi-agent dev-loop journaling (Codex CLI + Claude Code).
- **Part 3** — an MCP bridge so a user's own terminal-based coding agent (Claude Code, Codex CLI, Gemini CLI) can use Argus's vault as a tool source, independent of the in-app dashboard chat.

Key scope decisions made along the way:

- Non-Claude models include **both** local endpoints (Ollama) and **hosted APIs of open-weight models** (Groq, Together, Fireworks, DeepInfra, OpenRouter, etc.) — local-only would work *against* the low-end-hardware goal, since local inference needs more horsepower, not less. Hosted-API entries are clearly badged "notes leave your machine" in the UI so the privacy trade-off is opt-in and visible, not silent.
- A model must support **tool/function calling** to be usable — no degraded no-tools fallback, on any provider.
- Codex's and Gemini's *models* are just OpenAI-compatible chat-completion endpoints — they slot into the same adapter as any hosted model, no bespoke integration needed (Level 1).
- Separately, Claude Code / Codex CLI / Gemini CLI as full **agent harnesses** (Level 2) get supported via one standard MCP server Argus exposes — not by embedding each CLI's proprietary orchestration loop.
- The local model picker builds on **Ollama** rather than a bundled custom runtime — reuses Ollama's model library, quantization handling, and storage management.

PART 1 Provider-agnostic agent runtime

Chat, planner, and study generation. Covers local endpoints, hosted-API open-weight models, and Codex/Gemini as model choices (Level 1).

The core problem

`build_vault_tools()` and `build_propose_tools()` return `claude_agent_sdk`'s `@tool`-decorated objects — SDK-specific values, not portable data. Running the same tool-calling loop against an arbitrary OpenAI-compatible endpoint means tool definitions need to become plain data any backend can translate, plus a second execution engine that runs the completion+tool-call loop by hand — Claude's SDK does that loop internally; OpenAI-compatible endpoints have no equivalent orchestrator, so Argus has to run it itself.

Design

New module `backend/agent/adapters.py`:

```

@dataclass
class ToolSpec:
    name: str
    description: str
    parameters: dict[str, Any]          # JSON-schema properties
    handler: Callable[[dict], Awaitable[dict]]

class AgentAdapter(Protocol):
    async def run(
        self, system_prompt: str, user_message: str, tools: list[ToolSpec],
        max_turns: int,
    ) -> AsyncIterator[AgentEvent]:    # text deltas + usage
        ...

```

- **ClaudeSDKAdapter** — thin wrapper around today's `ClaudeSDKClient` code (unchanged behavior).
- **OpenAICompatAdapter** — new, using the `openai` Python client. Handles **all** non-Claude cases: local Ollama, hosted-API open-weight providers, and Codex/Gemini's own APIs — they're all OpenAI-compatible chat-completion + tool-calling surfaces. Runs the manual tool loop, accumulates streamed tool-call deltas by index, dispatches to `handler`.

Registry gains a real `key_ref` path. The existing (unused) `key_ref` field becomes real: hosted-API entries store their key via `keyring` (same pattern as the gcal/todoist connectors, invariant I4), referenced by name only in `models.json` — never the raw key. `ModelsPanel.tsx` gains a masked key input shown only for "hosted API" entries, and a **LOCAL / HOSTED** badge (extending today's LOCAL/API badge) so the privacy distinction is visible at a glance.

Model choice extends to planner + study generation: `run_planner()` and `agent_generate()` gain a `model` parameter with registry-default-when-unset behavior. The Study tab gets the same `ModelSelect` chat already uses.

Capability probe on registration, unchanged from before: `POST /api/models` sends one throwaway tool-calling request before persisting a non-Claude entry; rejects with 422 if the model doesn't return a valid tool call.

Critical files

File	Change
<code>backend/agent/adapters.py</code>	new — ToolSpec, AgentAdapter, ClaudeSDKAdapter, OpenAICompatAdapter, resolve_adapter
<code>backend/agent/runtime.py</code> , <code>planner.py</code> , <code>generate.py</code>	build ToolSpec lists, route through resolve_adapter; planner/generate gain a model param
<code>backend/config.py</code>	<code>key_ref</code> becomes load-bearing (keyring lookup on read)

<code>backend/system_api.py</code>	local-vs-hosted distinction, key storage, capability probe
<code>pyproject.toml</code>	add openai to core dependencies
<code>web/components/system/ModelsPanel.tsx</code> , <code>ModelSelect.tsx</code>	key input, LOCAL/HOSTED badges
<code>web/app/(dashboard)/study/**</code>	add ModelSelect
<code>web/lib/flags.ts</code>	flip localModels off preview

Testing

- Unit tests for OpenAICompatAdapter's tool-call loop (mocked server): no tool call, single call, multi-turn, max_turns exhaustion.
- `POST /api/models` tests: capability probe accept/reject; key stored via keyring, never written to models.json in plaintext.
- Existing pytest suite stays green with ClaudeSDKAdapter as default.
- Manual: local Ollama model — chat citations still resolve, `/plan tomorrow` still lands in Review. Hosted-API model (e.g. a Groq-served Llama) — same two checks, plus confirming the UI badges it as non-local.

PART 1b Hardware-aware local model picker

Recommend models that fit the user's machine; one-click install via Ollama.

Design

- `backend/agent/hardware.py` — probes total RAM and, where possible, GPU VRAM (`nvidia-smi --query-gpu=memory.total` on NVIDIA; other vendors/platforms fall back to "unknown," showing only CPU-safe recommendations).
- `backend/agent/model_catalog.py` — a curated static list of known Ollama-library models tagged with `min_ram_gb`, `min_vram_gb`, `size_gb`, `tool_calling: bool`. Only tool-calling models are listed at all.
- `GET /api/models/catalog` — the curated list, annotated per entry with a fit verdict (`fits` / `slow` / `insufficient`) against the detected hardware profile.
- `POST /api/models/install {name}` — shells out to `ollama pull <name>`, streaming progress back, then auto-registers the model on completion.
- **Install location:** Ollama owns its own model storage (`OLLAMA_MODELS` env var); `ollama pull` has no per-call destination flag. Argus surfaces Ollama's current storage path rather than promising a per-model custom path Ollama itself doesn't support — changing location is a one-time `OLLAMA_MODELS` setup step, documented, not a per-download choice.
- `argus doctor` gains an Ollama reachability check.

- **UI:** new "Local Models" browser on `/system` — hardware-fit badges, Download button with inline progress, installed models flow straight into the normal selector.

Spike required: cross-platform VRAM detection is inherently vendor-specific. Confirm what's reliably detectable on Windows (the primary target) before committing to a detection strategy for macOS/Linux.

Critical files

File	Change
<code>backend/agent/hardware.py</code> , <code>model_catalog.py</code>	new
<code>backend/system_api.py</code>	GET <code>/api/models/catalog</code> , POST <code>/api/models/install</code>
<code>backend/doctor.py</code>	Ollama check
<code>web/app/(dashboard)/system/**</code>	new Local Models browser

Testing

- Fit-verdict logic unit tested against synthetic hardware profiles (8GB RAM/no GPU, 16GB RAM/12GB VRAM, unknown GPU).
- `POST /api/models/install` tested against a mocked `ollama pull` (progress streaming, registry auto-registration on success, clean error surfacing on failure).
- Manual: on Ethan's actual machine, confirm the catalog's fit verdicts match reality and a real `ollama pull` completes end-to-end into the selector.

PART 2 Multi-agent dev-loop journaling

Rename `claude-integration/` → `agent-integration/`, extract the shared "append a session stub" PowerShell logic into `lib/write-session-stub.ps1`, add a `codex/` hook adapter alongside `claude/`. `setup.ps1` gains `-Agent claude|codex|all`.

Spike required: confirm what Codex CLI's session-lifecycle hooks actually support — context-injection parity with Claude Code's `SessionStart` hook is unconfirmed. Session-end journaling works regardless (pure logging, no injection needed).

Critical files

File	Change
<code>claude-integration/</code> → <code>agent-integration/</code>	git mv, preserves history

<code>agent-integration/lib/write-session-stub.ps1</code>	new, shared logic
<code>agent-integration/codex/hooks/*.ps1</code>	new, after the spike
<code>agent-integration/setup.ps1</code>	-Agent param
<code>README.md</code>	"Claude Code dev loop" → "Coding-agent dev loop"

PART 3 MCP bridge for external coding agents

Lets a user's own terminal-based Claude Code / Codex CLI / Gemini CLI use Argus's vault as a tool source — Level 2 integration, separate from the in-app dashboard chat.

Design

Part 1's refactor already decouples `search_vault` / `read_note` / `list_tasks` into plain `ToolSpec` s. Part 3 reuses those same `ToolSpec.handler` functions behind a standalone MCP server, instead of writing per-CLI bespoke integrations:

- `argus mcp-server` — new CLI subcommand, starts an MCP server (stdio transport — the standard local-attachment method for Claude Code/Codex CLI/Gemini CLI) exposing the same read-only tools chat uses today.
- **Scope, deliberately narrow for v1:** read-only tools only (`search_vault`, `read_note`, `list_tasks`). The `propose_*` planner tools are *not* exposed here — an external CLI silently queuing suggestions without the in-app planner's context would be a worse experience than not having it; revisit once the read-only path is proven.
- **Docs:** a "Connect your coding agent" section with three short config snippets — Claude Code (`claude mcp add argus -- argus mcp-server`), Codex CLI's MCP config, Gemini CLI's MCP config — mirroring the existing `claude mcp add obsidian` line already in the README, generalized.
- **Relationship to Part 2:** distinct feature. Part 2 journals sessions on the Argus repo itself; Part 3 gives any coding agent, on any project, read access to the user's vault. Both live under the "agent-integration" umbrella but don't share code.

Critical files

File	Change
<code>backend/agent/mcp_server.py</code>	new, wraps Part 1's ToolSpecs in an MCP server
<code>backend/cli.py</code>	<code>argus mcp-server</code> subcommand
<code>pyproject.toml</code>	add <code>mcp</code> (Python MCP SDK)
<code>README.md</code>	"Connect your coding agent" section

Testing

- MCP server unit tests: tool list matches chat's read-only set; a `search_vault` call through the MCP transport returns the same shape chat's in-process call does.
- Manual: attach Claude Code to `argus mcp-server`, ask a vault question from an unrelated project directory, confirm it calls `search_vault` and cites a real path. Repeat for Codex CLI / Gemini CLI once their MCP config format is confirmed (spike, same caveat as Part 2's Codex hook spike).

Delivery order

1. **Part 1** — adapter abstraction + local/hosted-API routing, chat first, then planner + study generation. Proves the core abstraction.
2. **Part 1b** — hardware detection + catalog + Ollama-backed install flow, on top of Part 1's registry.
3. **Part 3** — MCP bridge (reuses Part 1's ToolSpecs directly, cheap once Part 1 lands; read-only scope keeps it low-risk).
4. **Part 2** — dev-loop journaling rename + Codex adapter (independent of the other three; can run in parallel with any of them).
5. **Docs** — update README.md / docs/BUILD_STATE.md to reflect all four.

Verification (end-to-end)

- `pytest` and `ruff check` stay green throughout (regression proof on the Claude path).
- `npm run lint && npm run build` stay green; `npm run e2e` covers the Study-tab model selector and the Local Models browser.
- Live smoke test: local Ollama model, hosted-API model, and Codex/Gemini model each answer a real chat question with correct citations, and each successfully runs `/plan tomorrow` into the Review queue.
- Live smoke test: hardware picker's fit verdicts match reality on Ethan's machine; a recommended model installs end-to-end via one click.
- Live smoke test: Claude Code (and, once confirmed, Codex CLI / Gemini CLI) attached via `argus mcp-server` can search and cite the vault from an unrelated project directory.
- Live smoke test: a real Codex CLI session on this repo produces a journal stub matching Claude Code's format.